



FindTy

จัดทำโดย

นาย เอื้ออวัช เตชะวีรพงศ์ 1660702513

นาย เขมเดช เหนียวแน่น 1660702844

นาย วชิรวิทย์ ตันติพูลผล 1660704980

นางสาว กัญญ์วรา บ่นหา 1660707314

อาจารย์ผู้สอน

อาจารย์ ชนวีร์ พิพัฒน์กุล

ในรายวิชา CS356-Mobile Application Development I

หลักสูตรวิทยาศาสตรบัณฑิต

ภาคการศึกษาที่ 2 ปีการศึกษา 2568

ภาควิชาวิทยาการคอมพิวเตอร์

คณะเทคโนโลยีสารสนเทศและนวัตกรรม

มหาวิทยาลัยกรุงเทพ

ปีการศึกษา 2568

สารบัญ

1. ภาพรวมโปรแกรม	4
1.1 ชื่อโปรแกรม	4
1.2 วัตถุประสงค์	4
1.3 เทคโนโลยีที่ใช้	4
2. ฟังก์ชันการทำงานของโปรแกรม	5
2.1 ระบบผู้ใช้งาน (Authentication)	5
2.1.1 ฟังก์ชัน Login	5
2.1.2 ฟังก์ชัน Logout	5
2.1.3 ฟังก์ชันสมัครสมาชิก (Register)	6
2.2 ระบบห้องเล่นเกม (Party System)	7
2.2.1 ฟังก์ชันค้นหาและกรองเกม	7
2.2.2 ฟังก์ชันสร้างห้องเล่นเกม	8
2.2.3 ฟังก์ชันลบห้องเล่นเกม	9
2.2.4 ฟังก์ชันเข้าร่วมและออกจากห้อง	10
2.3 ระบบแชท (Chat System)	11
2.4 ระบบโปรไฟล์และรีวิว	12
2.4.1 ฟังก์ชันแก้ไขโปรไฟล์	12
2.4.2 ฟังก์ชันรีวิวผู้ใช้งาน	13
2.4.3 ฟังก์ชันเปลี่ยนรหัสผ่าน	14
3. สถาปัตยกรรมระบบ (System Architecture)	15
3.1 ภาพรวมสถาปัตยกรรม	15
3.2 โครงสร้างโปรเจกต์	16
4. ฐานข้อมูล (Firebase Backend)	17
4.1 Firebase Authentication	17
4.2 Firestore Database Schema	17
4.2.1 Collection: users	17
4.2.2 Collection: parties	18
4.2.3 Collection: games	19
5. ระบบความปลอดภัย (Firestore Security Rules)	20
5.1 ภาพรวมสิทธิ์การเข้าถึง	20
5.2 หลักการ Defense in Depth	20
5.3 จุดเด่นด้านความปลอดภัย	20

6. Package และ Dependencies	21
7. ตัวอย่าง Data Flow การทำงาน	22
7.1 Flow การสร้างดี	22
7.2 Flow การส่งข้อความแชท	22
8. สรุป	23

1. ภาพรวมโปรแกรม

1.1 ชื่อโปรแกรม

แอปพลิเคชันFindTy(ไฟต์ตี้)

1.2 วัตถุประสงค์

Findtyเป็นแอปพลิเคชันมือถือที่ช่วยให้ผู้เล่นเกมออนไลน์สามารถค้นหาและเชื่อมต่อกับเพื่อนร่วมเล่นเกม

(Gaming Party) ได้แบบเรียลไทม์ โปรแกรมมีวัตถุประสงค์หลักดังนี้

- ช่วยให้ผู้เล่นสามารถสร้างห้อง (ตี้) เพื่อหาเพื่อนเล่นเกมได้อย่างรวดเร็ว
- รองรับการแชทภายในห้องเกมแบบเรียลไทม์
- ให้ผู้ใช้สามารถรีวิวและประเมินเพื่อนร่วมเล่นเกมได้
- รองรับหลายเกมพร้อมกันโดยสามารถเพิ่มเกมใหม่ได้โดยไม่ต้องแก้ไขโค้ด

1.3 เทคโนโลยีที่ใช้

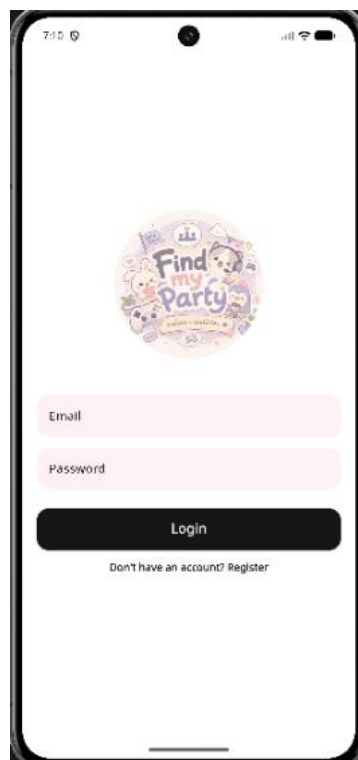
เทคโนโลยี	รายละเอียด
ภาษาโปรแกรม	Dart
Framework	Flutter (Cross-platform Mobile)
Database	Cloud Firestore (NoSQL Document Database)
Authentication	Firebase Authentication (Email/Password)
IDE	Android Studio
Version Control	Git

2. ฟังก์ชันการทำงานของโปรแกรม

2.1 ระบบผู้ใช้งาน (Authentication)

2.1.1 ฟังก์ชัน Login

ผู้ใช้งานสามารถเข้าสู่ระบบด้วย Email และ Password โดยระบบจะตรวจสอบข้อมูลผ่าน Firebase Authentication เมื่อ Login สำเร็จ แอปจะพาผู้ใช้ไปยังหน้าเลือกเกม (Game Selection) อัตโนมัติ โดยใช้ `authStateChanges()` เป็น Stream ฟังสถานะการ Login ตลอดเวลา



2.1.2 ฟังก์ชัน Logout

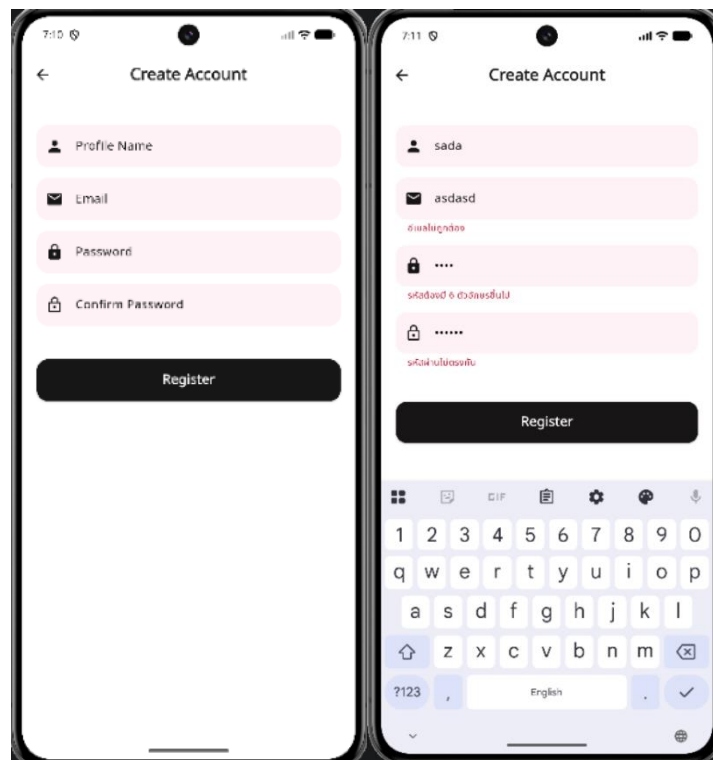
ผู้ใช้งานสามารถออกจากระบบได้จากปุ่ม Logout ในหน้าเลือกเกม เมื่อ Logout สำเร็จ แอปจะพาผู้ใช้กลับไปยังหน้า Login อัตโนมัติโดยไม่ต้อง navigate เอง เนื่องจากใช้ `StreamBuilder` ฟัง `authStateChanges()`



2.1.3 ฟังก์ชันสมัครสมาชิก (Register)

ผู้ใช้กรอก Profile Name, Email และ Password เพื่อสมัครสมาชิก ระบบจะทำงานตามลำดับดังนี้

1. สร้าง Auth user ผ่าน AuthService.register()
2. สร้าง document users/{uid} ใน Firestore ผ่าน UserService.createOrUpdateProfile()
3. Logout อัตโนมัติ เพื่อให้ผู้ใช้ Login ด้วยตนเอง (UX ที่ดีกว่าการ auto-login ทันที)
4. กลับสู่หน้า Login

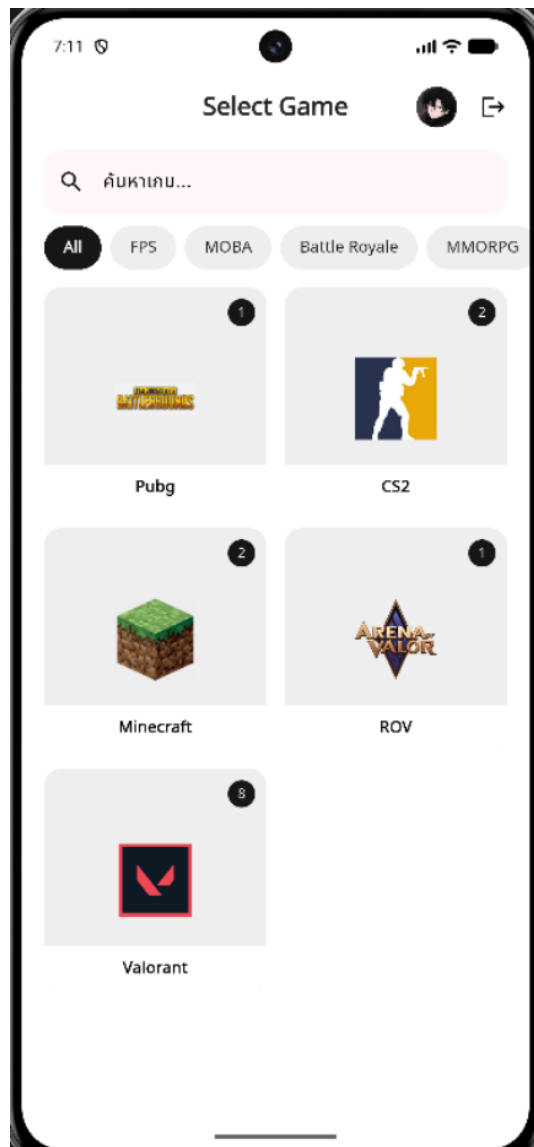


2.2 ระบบห้องเล่นเกม (Party System)

2.2.1 ฟังก์ชันค้นหาและกรองเกม

หน้า Game Selection แสดงรายชื่อเกมทั้งหมดในรูปแบบ Grid View พร้อมฟีเจอร์ดังนี้

- ช่องค้นหาชื่อเกม (Search Bar) แบบ real-time filtering
- ปุ่ม Filter แยกหมวดหมู่เกม เช่น FPS, MOBA, RPG, Strategy
- แต่ละการ์ดเกมแสดงจำนวนห้อง (ตู้) ที่เปิดอยู่แบบ real-time
- ข้อมูลเกมดึงมาจาก Firestore ทำให้สามารถเพิ่มเกมใหม่ได้โดยไม่ต้องแก้โค้ด

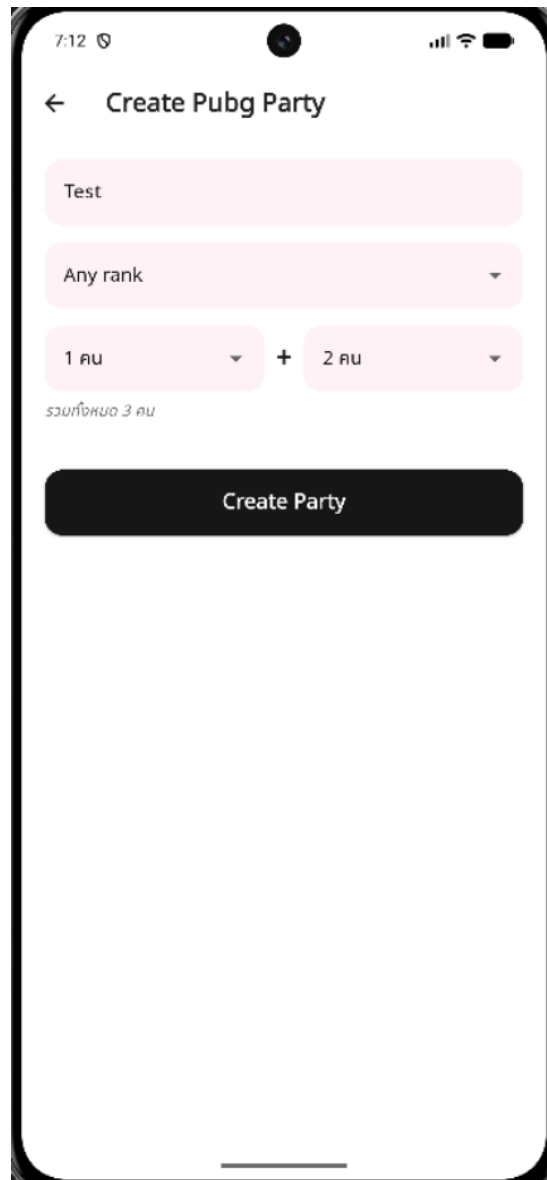


2.2.2 ฟังก์ชันสร้างห้องเล่นเกม

ผู้ใช้ที่ Login แล้วสามารถสร้างห้อง (ตี้) ได้โดยกด FAB (+) ในหน้ารายการตี้

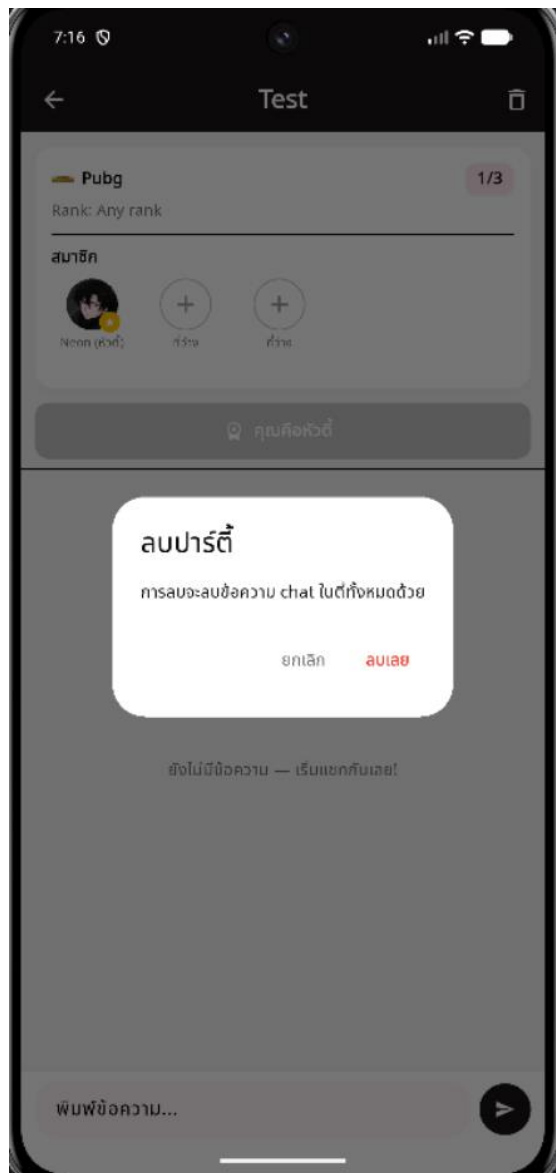
ฟอร์มสร้างตี้มีรูปแบบที่ปรับเปลี่ยนตามประเภทเกม (Adaptive UI) ดังนี้

- ชื่อห้อง (Title)
- Rank dropdown — แสดงเฉพาะเกมที่มีระบบ rank
- Role — ถ้าเกมอนุญาต multi-role จะใช้ FilterChip เลือกได้หลายตำแหน่ง
- จำนวนสมาชิก — กำหนดได้ตามขนาดสูงสุดของเกม



2.2.3 ฟังก์ชันลบห้องเล่นเกม

เจ้าของห้อง (Owner) สามารถลบห้องได้จากปุ่มในหน้ารายละเอียดห้อง ระบบจะแสดง Confirmation Dialog ก่อนลบ และใช้ WriteBatch ลบ Party document พร้อมกับ Messages ทั้งหมดใน subcollection ในการ commit เดียว เพื่อความสมบูรณ์ของข้อมูล

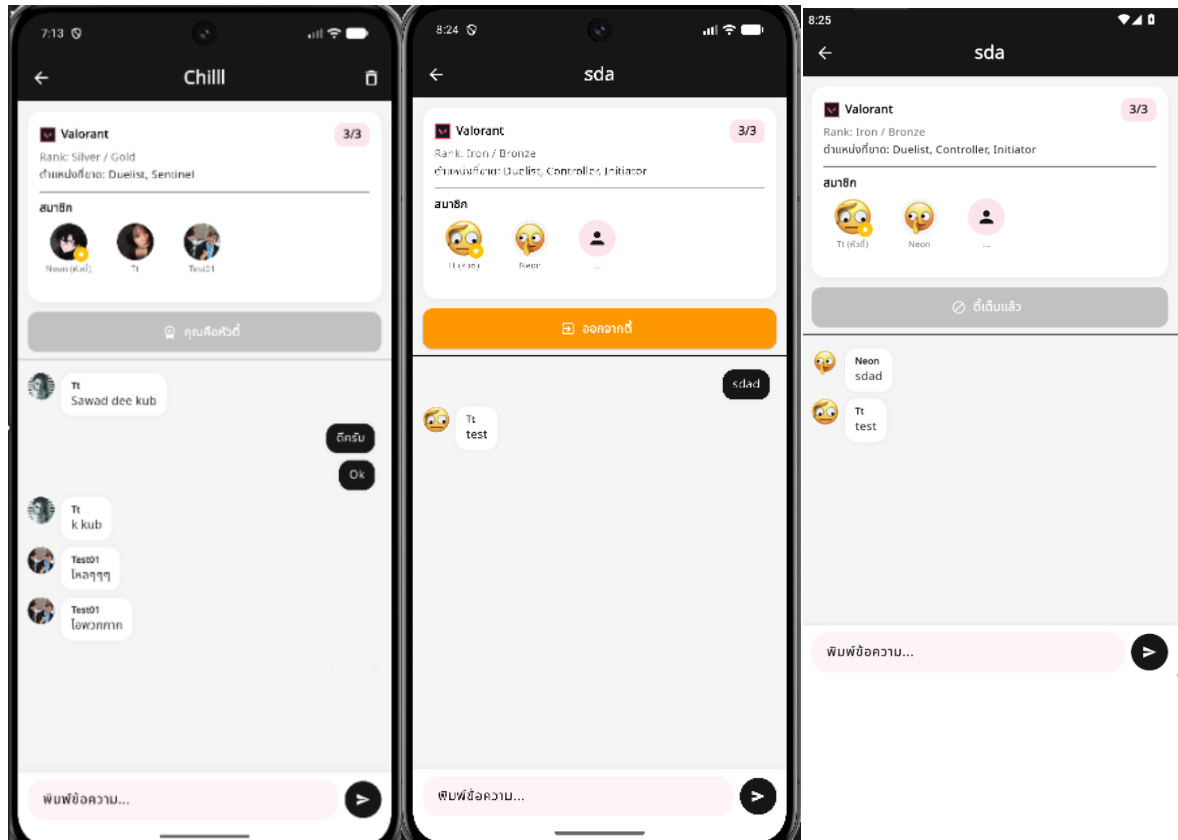


2.2.4 ฟังก์ชันเข้าร่วมและออกจากห้อง

ผู้ใช้สามารถเข้าร่วม (Join) หรือออกจาก (Leave) ห้องได้ โดยระบบใช้ Firestore Transaction

เพื่อป้องกัน Race Condition กรณีที่มีผู้ใช้หลายคนกด Join พร้อมกัน ปุ่มจะเปลี่ยนสถานะตาม 4

รูปแบบ ได้แก่ เจ้าของห้อง / เป็นสมาชิกอยู่แล้ว / ห้องเต็ม / สามารถเข้าร่วมได้



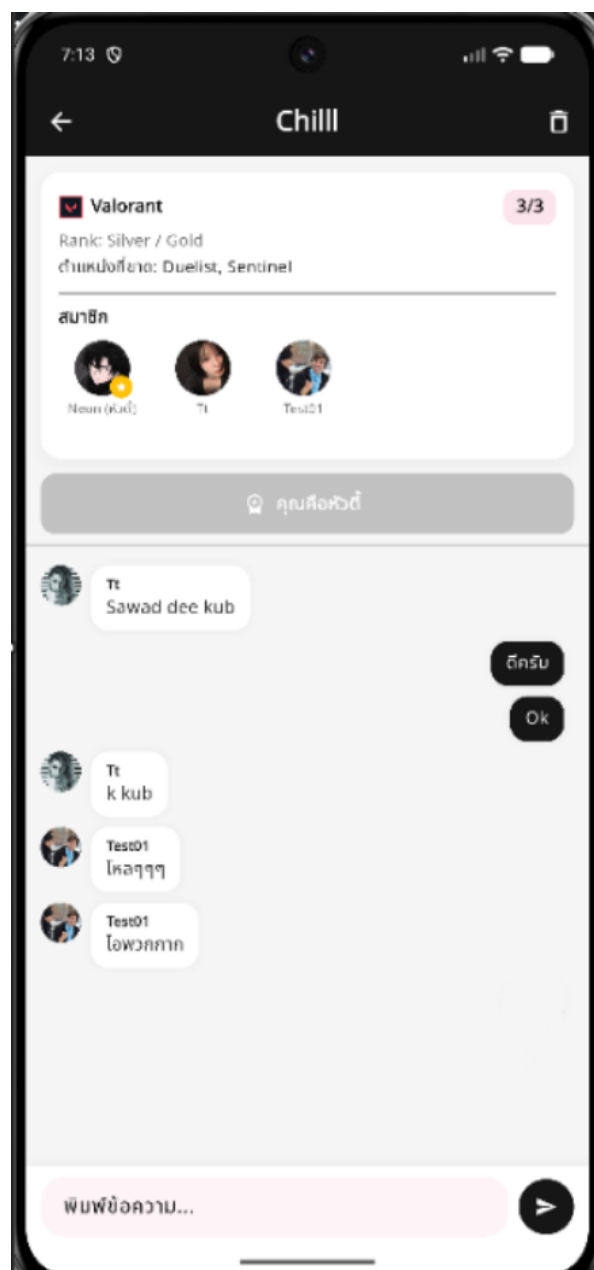
2.3 ระบบแชท (Chat System)

ผู้ใช้ที่อยู่ในห้องสามารถพิมพ์ข้อความและส่งหากันได้แบบ Real-time

โดยข้อความจะปรากฏบนหน้าจอของทุกคนในห้องทันทีโดยไม่ต้องรีเฟรช ข้อความแต่ละอัน (Message)

เก็บข้อมูล ได้แก่ ผู้ส่ง (senderId, senderName, senderAvatar), เนื้อหาข้อความ (text) และเวลาที่ส่ง

(createdAt) โดยเก็บใน Firestore subcollection parties/{partyId}/messages

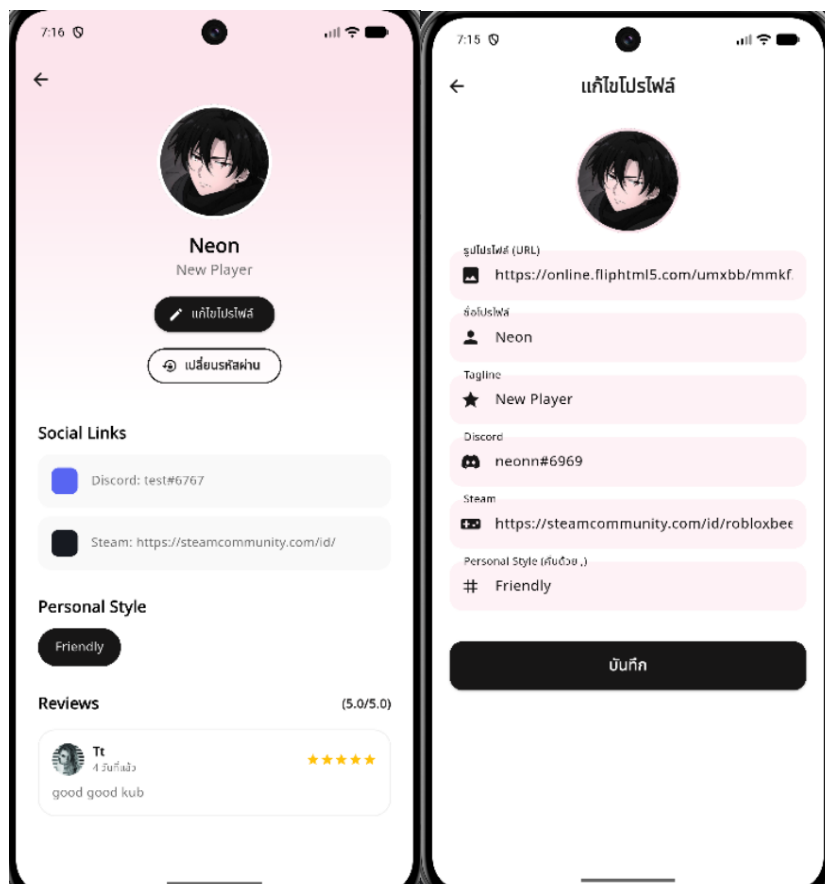


2.4 ระบบโปรไฟล์และรีวิว

2.4.1 ฟังก์ชันแก้ไขโปรไฟล์

ผู้ใช้สามารถแก้ไขข้อมูลส่วนตัวได้ ประกอบด้วย

- ชื่อผู้ใช้ (Username)
- รูปโปรไฟล์ (Avatar URL) — พร้อม Preview แบบ Live
- คำอธิบายตัวเอง (Tagline)
- ข้อมูล Social เช่น Discord Tag และ Steam ID
- Traits (คำบอกสไตล์การเล่น เช่น Aggressive, Friendly) — กรอกแบบ comma-separated

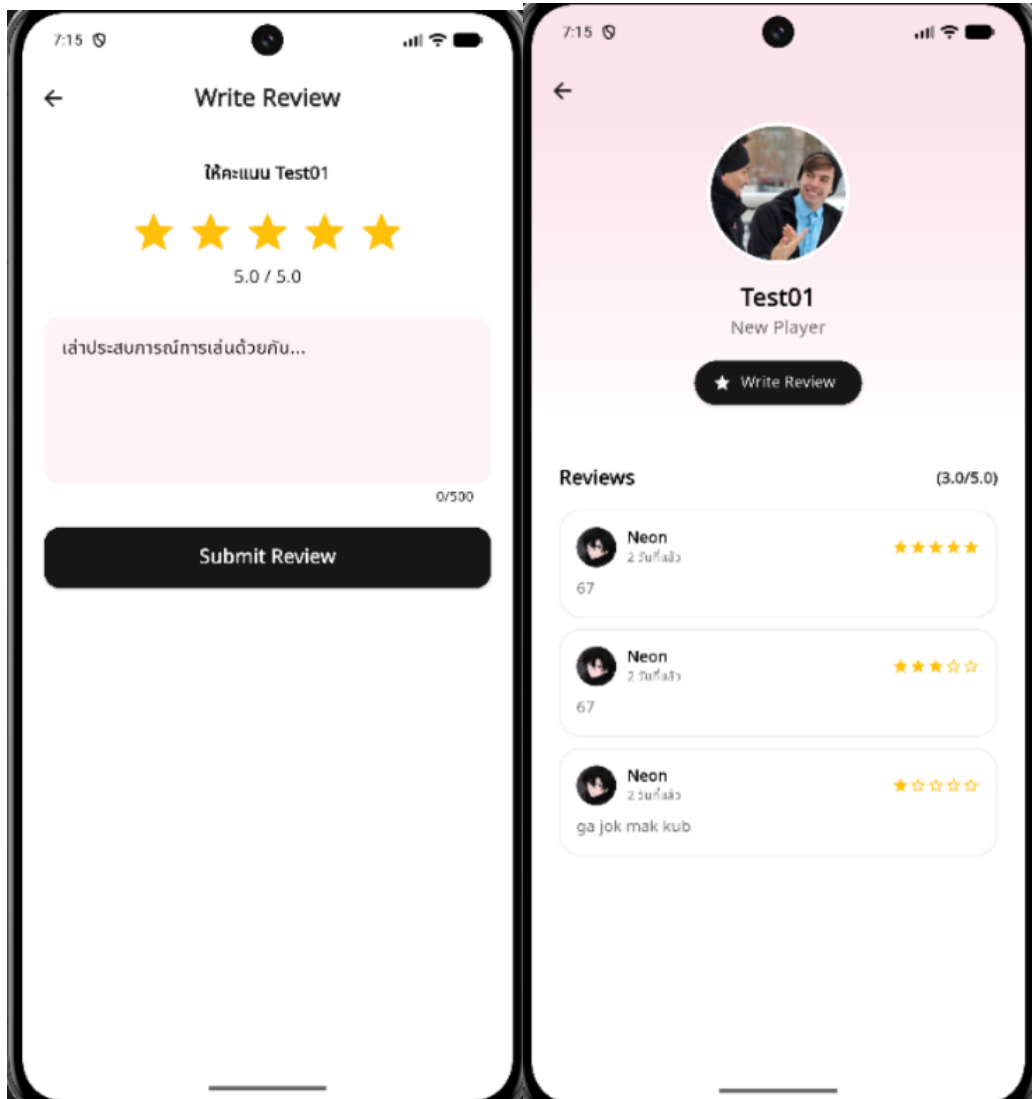


2.4.2 ฟังก์ชันรีวิวใช้งาน

ผู้ใช้สามารถเขียนรีวิวให้กับผู้อื่นได้โดยให้คะแนนดาว 1-5 ดาว พร้อมข้อความรีวิว

ระบบมีการป้องกันการรีวิวตัวเอง ทั้งในฝั่ง UI (ซ่อนปุ่ม Write Review ในโปรไฟล์ตนเอง) และฝั่ง

Firestore Security Rules รีวิวที่เขียนแล้วไม่สามารถแก้ไขหรือลบได้



2.4.3 ฟังก์ชันเปลี่ยนรหัสผ่าน

ผู้ใช้สามารถเปลี่ยนรหัสผ่านได้โดยกรอกรหัสผ่านเดิม รหัสผ่านใหม่ และยืนยันรหัสผ่านใหม่ ระบบจะ Reauthenticate ก่อนเปลี่ยน Password เสมอ

7:16

← เปลี่ยนรหัสผ่าน

รหัสผ่านปัจจุบัน

กรอกรหัสผ่านปัจจุบัน

รหัสผ่านใหม่

ต้องมียาวน้อย 6 ตัวอักษร

ยืนยันรหัสผ่านใหม่

กรอกรหัสผ่านใหม่อีกครั้ง

บันทึกรหัสผ่านใหม่

3. สถาปัตยกรรมระบบ (System Architecture)

3.1 ภาพรวมสถาปัตยกรรม

แอป Findty ใช้สถาปัตยกรรม 3 ชั้น (3-Layer Architecture)

โดยแต่ละชั้นมีหน้าที่ชัดเจนและแยกออกจากกัน

ชั้น	หน้าที่
Screens (UI Layer)	หน้าจอที่ผู้ใช้เห็นและโต้ตอบ ประกอบด้วย 10 หน้าจอหลัก
Services Layer	เชื่อมต่อ Firebase โดย Screens จะเรียกใช้ผ่าน Services เท่านั้น ไม่ติดต่อ Firebase โดยตรง
Models Layer	Data Classes ที่ใช้รับ-ส่งข้อมูลระหว่างชั้น
Firebase Backend	Authentication และ Cloud Firestore สำหรับจัดเก็บข้อมูลทั้งหมด

หลักการสำคัญ: Screens ไม่คุยกับ Firebase ตรงๆ — ผ่าน Services เท่านั้น ทำให้ถ้าจะเปลี่ยน Backend ในอนาคตสามารถแก้ไขแค่ Services โดยไม่กระทบ UI

3.2 โครงสร้างโปรเจกต์

โครงสร้างโฟลเดอร์ของโปรเจกต์แบ่งตามหน้าที่ดังนี้

```
lib/
├─ main.dart                ← entry point
├─ firebase_options.dart    ← Firebase API keys
├─ constants.dart          ← ค่าสีของแอป
├─ config/
│   └─ game_config.dart    ← model + parser ของเกม
├─ models/                 ← Data classes ทั้งหมด
│   ├── party_model.dart
│   ├── user_profile_model.dart
│   ├── review_model.dart
│   └─ message_model.dart
├─ services/               ← Firestore/Auth wrappers
│   ├── auth_service.dart
│   ├── user_service.dart
│   ├── party_service.dart
│   ├── chat_service.dart
│   └─ game_service.dart
└─ screens/                ← UI ทุกหน้า
    ├── login_screen.dart
    ├── register_screen.dart
    ├── game_selection_screen.dart
    ├── party_feed_screen.dart
    ├── create_party_screen.dart
    ├── party_detail_screen.dart
    ├── profile_screen.dart
    ├── edit_profile_screen.dart
    ├── write_review_screen.dart
    └─ change_password_screen.dart
```


4. ฐานข้อมูล (Firebase Backend)

4.1 Firebase Authentication

ใช้ระบบ Email / Password Authentication ของ Firebase เมื่อสมัครสมาชิกสำเร็จจะได้ UID (User ID) ซึ่งเป็น string ไม่ซ้ำ ใช้เป็น Primary Key ของผู้ใช้ทุกทีในแอป

4.2 Firestore Database Schema

Firestore จัดเก็บข้อมูลในรูปแบบ Document-based NoSQL แบ่งออกเป็น 3 Collection หลัก

4.2.1 Collection: users

Field	Type	คำอธิบาย
uid	string	User ID จาก Firebase Auth (Primary Key)
username	string	ชื่อแสดงผล
email	string	Email ที่ใช้สมัคร
avatarUrl	string	URL รูปโปรไฟล์
discordTag	string	Discord Username
steamId	string	Steam ID
tagline	string	คำอธิบายตัวเอง
traits[]	array	สไตล์การเล่น เช่น Aggressive, Friendly
createdAt	timestamp	วันที่สมัคร

Subcollection: users/{uid}/reviews/{reviewId}

Field	Type	คำอธิบาย
reviewerUid	string	UID ของผู้เขียนรีวิว
reviewerName	string	ชื่อผู้เขียนรีวิว (snapshot)
reviewerAvatar	string	รูปผู้เขียนรีวิว (snapshot)
rating	number	คะแนน 1-5 ดาว
text	string	เนื้อหารีวิว
createdAt	timestamp	วันที่เขียน

4.2.2 Collection: parties

Field	Type	คำอธิบาย
id	string	Party ID (Document ID)
title	string	ชื่อห้อง
gameId	string	ID ของเกม
rank	string?	Rank ของห้อง (optional)
role	string?	Role ที่กำหนด (optional)
current	number	จำนวนสมาชิกปัจจุบัน
max	number?	จำนวนสมาชิกสูงสุด
ownerId	string	UID เจ้าของห้อง
ownerName	string	ชื่อเจ้าของห้อง (denormalized)
leadAvatar	string	รูปเจ้าของห้อง (denormalized)
memberIds[]	array	UID สมาชิกทุกคน (index 0 = owner)
createdAt	timestamp	วันที่สร้างห้อง

Subcollection: parties/{partyId}/messages/{msgId}

Field	Type	คำอธิบาย
senderId	string	UID ผู้ส่ง
senderName	string	ชื่อผู้ส่ง (snapshot)
senderAvatar	string	รูปผู้ส่ง (snapshot)
text	string	เนื้อหาข้อความ
createdAt	timestamp	เวลาที่ส่ง

4.2.3 Collection: games

Field	Type	คำอธิบาย
displayName	string	ชื่อเกม
iconUrl	string	URL รูป icon เกม
category	string	หมวดหมู่ เช่น FPS, MOBA, RPG
hasRank	boolean	เกมมีระบบ rank หรือไม่
ranks[]	array	รายชื่อ rank ทั้งหมด
hasRole	boolean	เกมมีระบบ role หรือไม่
roles[]	array	รายชื่อ role ทั้งหมด
allowMultiRole	boolean	เลือก role ได้หลายอันหรือไม่
maxPartySize	number?	จำนวนผู้เล่นสูงสุด (null = ไม่จำกัด)
memberLabel	string	ป้าย เช่น 'คน', 'ผู้เล่น'

5. ระบบความปลอดภัย (Firestore Security Rules)

5.1 ภาพรวมสิทธิการเข้าถึง

Resource	Read	Create	Update	Delete
games/*	ทุกคน	ไม่ได้	ไม่ได้	ไม่ได้
parties/*	ทุกคน	Login	Login	เจ้าของเท่านั้น
parties/*/messages/*	Login	Login	—	Login
users/*	ทุกคน	เจ้าของ	เจ้าของ	—
users/*/reviews/*	ทุกคน	ไม่ใช่เจ้าของ	ไม่ได้	ไม่ได้

5.2 หลักการ Defense in Depth

ระบบทำการ Validate ข้อมูล 2 ชั้น เพื่อความปลอดภัย

- ชั้น UI : ป้องกันผู้ใช้ทั่วไปไม่ให้กดผิด เช่น ซ่อนปุ่ม 'Write Review' ในโปรไฟล์ตนเอง
- ชั้น Firestore Rules : ป้องกัน Hacker ที่พยายาม Bypass UI โดยเรียก API ตรง เช่น Rules จะบล็อกการ Review ตัวเอง

5.3 จุดเด่นด้านความปลอดภัย

- รีวิวที่เขียนแล้วไม่สามารถแก้ไขหรือลบได้ ป้องกันการปั่นบังคับให้ลบรีวิว
- เฉพาะเจ้าของห้องเท่านั้นที่ลบได้
- ใช้ Firestore Transaction ป้องกัน Race Condition ตอน Join/Leave ห้อง

6. Package และ Dependencies

Package	หน้าที่	ใช้ในไฟล์
firebase_core	เริ่มต้นและเชื่อมต่อ Firebase	main.dart
firebase_auth	ระบบ Login / Register	auth_service.dart
cloud_firestore	ฐานข้อมูล NoSQL	ทุก Service
google_fonts	ฟอนต์ Noto Sans Thai	main.dart (theme)
flutter_localizations	รองรับ Locale ภาษาไทย	main.dart
flutter_rating_bar	UI ดาว 1-5 ดวง	write_review_screen.dart

7. ตัวอย่าง Data Flow การทำงาน

7.1 Flow การสร้างดี

ขั้นที่	การทำงาน
1	User กด FAB (+) ในหน้า PartyFeedScreen
2	Navigator push ไปยัง CreatePartyScreen
3	User กรอกฟอร์ม (ชื่อห้อง, rank, role, จำนวนคน) แล้วกด Create
4	ระบบเรียก AuthService.instance.currentUid เพื่อดึง UID
5	เรียก UserService.getProfile(uid) เพื่อ Snapshot ชื่อ+รูป
6	สร้าง PartyModel พร้อม memberIds: [uid]
7	เรียก PartyService.createParty(party) → เขียนลง Firestore
8	Firestore บันทึก Document → Trigger Snapshot ของทุก Client ที่ฟังอยู่
9	StreamBuilder ใน PartyFeedScreen ของทุกคน Rebuild → ดึงใหม่ไฟล์ทันที
10	Navigator.pop() กลับหน้า Feed

7.2 Flow การส่งข้อความแชท

ขั้นที่	การทำงาน
1	User พิมพ์ข้อความใน TextField แล้วกดปุ่มส่ง
2	ระบบเรียก UserService.getProfile(myUid) เพื่อดึงชื่อ+รูป
3	สร้าง Message object พร้อมข้อมูลผู้ส่งและเนื้อหา
4	เรียก ChatService.sendMessage(partyId, msg) → เขียนลง Firestore Subcollection
5	StreamBuilder ใน Chat List ของทุกคนในห้อง Rebuild → Bubble ใหม่ไฟล์ + เลื่อนลงล่าง

8. สรุป

Findty (ไฟด์ตี้) เป็นแอปพลิเคชันมือถือที่พัฒนาด้วย Flutter และ Firebase

เพื่อช่วยให้ผู้เล่นเกมออนไลน์สามารถหาเพื่อนเล่นเกม (Gaming Party) ได้แบบเรียลไทม์

แอปมีฟังก์ชันครบครันตั้งแต่ระบบสมาชิก การสร้างและจัดการห้องเล่นเกม ระบบแชทเรียลไทม์

ไปจนถึงระบบรีวิวผู้ใช้งาน

จุดเด่นของระบบ

- Real-time Update ทุกหน้าจอด้วย Firestore StreamBuilder โดยไม่ต้องรีเฟรชด้วยตนเอง
- สถาปัตยกรรม 3 ชั้นที่แยก UI, Business Logic และ Database ออกจากกันชัดเจน
- ระบบความปลอดภัย 2 ชั้น (UI Validation + Firestore Security Rules)
- Adaptive UI ที่ปรับฟอร์มตามประเภทเกมอัตโนมัติ ทำให้เพิ่มเกมใหม่ได้ง่ายโดยไม่แก้โค้ด
- Firestore Transaction ป้องกัน Race Condition ในการเข้าร่วม/ออกจากห้อง